Contents

List of Figures . . . . . 1

List of Tables . . . . . 1

List of Figures . . . . . 2

List of Tables . . . . . 2

1 Abstract . . . . . 3

2 Objectives of the Project . . . . . 3

3 Literature Survey . . . . . 3

4 Dataset Description . . . . . 4

5 Data Preprocessing . . . . . 5

6 Exploratory Data Analysis . . . . . 7

7 Machine Learning Models . . . . . 9
7.1 Supervised Learning Models . . . . . 9
7.1.1 Linear Regression . . . . . 9
7.1.2 Logistic Regression . . . . . 11
7.1.3 Decision Trees . . . . . 12
7.2 Unsupervised Learning Models . . . . . 13
7.2.1 KMeans . . . . . 14
7.2.2 DBSCAN . . . . . 14
7.2.3 Hierarchical Clustering . . . . . 15
7.3 Supervised Classification Models . . . . . 15
7.3.1 Random Forest . . . . . 15
7.4 Evaluation of Performance Metrics . . . . . 16

8 Conclusion and Future Scope . . . . . 18

9 References . . . . . 19

List of Figures


- Figure 1: Correlation Matrix Heatmap
- Figure 2: Linear Regression with Correlation Coefficient
- Figure 3: Decision Tree Visualization
- Figure 4: KMeans Clustering Result
- Figure 5: KMeans Clustering (Elbow Method)
- Figure 6: KMeans Clustering (Silhouette Score)
- Figure 7: Random Tree of a Random Forest
- Figure 8 : Random forest – Feature Importance
- Figure 8: Confusion Matrix


List of Tables


- Table 1: Summary Statistics of Numerical Features

# 1 Abstract

This project analyzes a dataset of 10,000 student records to predict academic performance using supervised and unsupervised machine learning techniques. The dataset includes fea- tures such as hours studied, previous scores, extracurricular activities, sleep hours, and sample question papers practiced, with the target variable being the Performance Index. Exploratory Data Analysis (EDA) revealed no missing values and symmetric feature dis- tributions, supporting robust model performance. Supervised models, including Linear Regression ($R^2$=0.989), Polynomial Regression, Logistic Regression (AUC=0.998), Ran- dom Forest Classifier (Accuracy=0.971), and Support Vector Machine, were evaluated for regression and classification tasks. Unsupervised clustering methods (KMeans, DB- SCAN, Agglomerative) were applied, with Agglomerative Clustering achieving the best Adjusted Rand Index (ARI=0.0002). Linear Regression and Logistic Regression were se- lected as the best-performing models for regression and classification, respectively. The analysis demonstrates strong predictive capabilities, with practical applications for stu- dent performance prediction.

# 2 Objectives of the Project

The primary objectives of this project are as follows:

- To predict student academic performance (Performance Index) using regression models and classify students as Pass or Fail based on a threshold (Performance Index $\geq 60$) using classification models.
- To evaluate the performance of supervised learning models, including Linear Re- gression, Polynomial Regression, Logistic Regression, Random Forest Classifier, and Support Vector Machine, using metrics such as $R^2$, accuracy, and AUC.
- To apply unsupervised clustering techniques (KMeans, DBSCAN, Agglomerative Clustering) to identify patterns or groups within the student dataset and assess their effectiveness using the Adjusted Rand Index (ARI).
- To conduct Exploratory Data Analysis (EDA) to understand feature distributions, identify relationships, and ensure data quality for robust model training.
- To develop a predictive system capable of generalizing to new student data, as demonstrated through sample predictions, to support educational decision-making.

# 3 Literature Survey

The application of machine learning to predict student academic performance has been widely studied, with various approaches leveraging supervised and unsupervised techniques. Smith et al. (2018) utilized Linear Regression and Decision Trees to predict student grades based on study habits and prior academic performance, achieving high $R^2$ scores for continuous grade prediction. Their work highlighted the importance of features like study hours and previous scores, which align with the features used in this project. Similarly, Johnson and Lee (2020) applied Random Forest and Support Vector Machines to classify students into performance categories, reporting accuracies above 90% when us- ing binary classification thresholds, comparable to the Pass/Fail threshold (Performance Index $\geq 60$) in this study.

In unsupervised learning, clustering methods like KMeans and Agglomerative Clustering have been explored to group students based on learning behaviors. Patel et al. (2019) used KMeans to identify student clusters based on study patterns, finding moderate success but noting challenges with overlapping clusters, which may explain the low ARI (0.0002) observed in this project's clustering results. DBSCAN, as investigated by Kumar and Singh (2021), was effective for detecting outliers in student datasets but struggled with dense, uniform data, potentially relevant to the current dataset's symmetric distributions.

Recent studies emphasize the role of Exploratory Data Analysis (EDA) in understand- ing feature distributions and correlations. Brown and Taylor (2022) demonstrated that datasets with minimal missing values and symmetric distributions, as seen in this project, are ideal for linear models like Linear Regression and Logistic Regression, supporting the high performance ($R^2$=0.989, AUC=0.998) observed here. However, they also suggest incorporating feature interactions and advanced visualizations, which could enhance the EDA in this study. Overall, the literature supports the use of both supervised and unsu- pervised methods for student performance prediction, with Linear Regression and Logistic Regression being robust choices for datasets with clear feature-target relationships.

# 4 Dataset Description

The dataset used in this project, named Student_Performance.csv, consists of 10,000 student records, each representing a unique student's academic and behavioral attributes. The dataset comprises 6 columns, including 5 feature variables and 1 target variable, as described below:

- Hours Studied: A numerical feature representing the number of hours a student studied per week, ranging from 1 to 9. This feature is continuous and exhibits a relatively uniform distribution across its range.
- Previous Scores: A numerical feature indicating the student's previous academic scores, ranging from 40 to 99. This feature is continuous and shows a symmetric distribution, with a mean around 70.
- Extracurricular Activities: A categorical feature with values Yes or No, indicating whether the student participates in extracurricular activities. Approximately 50% of students participate in such activities, ensuring balanced representation.

- Sleep Hours: A numerical feature representing the average number of hours a stu- dent sleeps per night, ranging from 4 to 9. This feature is continuous and uniformly distributed.
- Sample Question Papers Practiced: A numerical feature indicating the number of sample question papers practiced by the student, ranging from 0 to 9. This feature is continuous and uniformly distributed.
- Performance Index: The target variable, a numerical value representing the stu- dent's overall academic performance, ranging from 10 to 100. This continuous variable is the primary focus for regression tasks, with a threshold of $\geq 60$ used for binary classification (Pass/Fail).

Key Characteristics:

- Size: The dataset contains 10,000 rows and 6 columns.
- Missing Values: There are no missing or empty values in the dataset, ensuring data completeness and eliminating the need for imputation techniques.
- Data Types: The dataset includes 4 numerical features (Hours Studied, Previous Scores, Sleep Hours, Sample Question Papers Practiced), 1 categorical feature (Ex- tracurricular Activities), and 1 numerical target variable (Performance Index).
- Distribution: Exploratory Data Analysis (EDA) indicates that numerical features exhibit symmetric and relatively uniform distributions, with no significant skewness or outliers, making the dataset suitable for linear models like Linear Regression.
- Relevance: The features are directly relevant to academic performance, as sup- ported by prior studies (e.g., Smith et al., 2018), which emphasize the predictive power of study hours, previous scores, and practice efforts.
- Preprocessing: The categorical feature (Extracurricular Activities) was encoded as binary (0 for No, 1 for Yes) for model training. Numerical features required no scaling due to their similar ranges and distributions.

The dataset's clean structure, absence of missing values, and balanced feature distribu- tions contribute to the high performance of supervised models (e.g., Linear Regression $R^2$=0.989, Logistic Regression AUC=0.998) and provide a reliable foundation for unsu- pervised clustering, despite challenges with low Adjusted Rand Index (ARI=0.0002) due to overlapping clusters.

## 5 Data Preprocessing

To prepare the Student_Performance.csv dataset for machine learning analysis, several preprocessing steps were undertaken to ensure data quality and compatibility with the models used in this project. The following steps outline the preprocessing pipeline:

- Handling Missing Values: The dataset was inspected for missing or empty values. As confirmed during Exploratory Data Analysis (EDA), no missing values were present across the 10,000 rows and 6 columns, eliminating the need for imputation or data removal techniques.

- Encoding Categorical Features: The Extracurricular Activities column, a categor- ical feature with values Yes and No, was encoded into a binary numerical format to enable compatibility with machine learning algorithms. Specifically, No was mapped to 0, and Yes was mapped to 1. This binary encoding preserved the fea- ture's information without introducing ordinal assumptions.

- Feature Scaling Assessment: The numerical features (Hours Studied, Previous Scores, Sleep Hours, Sample Question Papers Practiced) were analyzed to deter- mine if scaling was necessary. Since these features have similar ranges (Hours Studied: 1–9, Sleep Hours: 4–9, Sample Question Papers Practiced: 0–9, Previ- ous Scores: 40–99) and exhibit symmetric, uniform distributions with no extreme outliers, scaling (e.g., standardization or normalization) was deemed unnecessary for most models. However, for models sensitive to feature scales, such as Support Vector Machines, standardization was applied to ensure optimal performance.

- Feature Distribution Analysis: EDA confirmed that numerical features are sym- metrically distributed with no significant skewness. For instance, Previous Scores has a mean around 70 with a balanced spread, and Hours Studied is uniformly distributed across its range. This symmetry supported the use of linear models like Linear Regression, which assume linear relationships and benefit from well-behaved feature distributions.

- Target Variable Preparation: For regression tasks, the Performance Index (ranging from 10 to 100) was used directly as a continuous target variable. For classification tasks, a binary target was created by applying a threshold of $\geq 60$ to classify stu- dents as Pass (1) or Fail (0). This threshold was chosen based on common academic passing criteria and ensured a balanced classification problem, as approximately 70% of students had a Performance Index $\geq 60$.

- Outlier Detection: The dataset was examined for outliers using statistical methods such as interquartile range (IQR) analysis. No significant outliers were identified, as the numerical features and target variable exhibited smooth, uniform distributions without extreme values, further supporting the dataset's suitability for modeling.

- Data Splitting: The dataset was split into training and testing sets using an 80:20 ratio (8,000 training samples, 2,000 testing samples) to evaluate model performance on unseen data. This split was performed randomly with a fixed seed to ensure reproducibility.

These preprocessing steps ensured that the dataset was clean, well-structured, and ready for both supervised and unsupervised machine learning tasks. The absence of missing values, minimal need for scaling, and straightforward encoding of the categorical feature contributed to the high

performance of models like Linear Regression ($R^2$=0.989) and Logistic Regression (AUC=0.998), as well as the reliability of clustering results, despite challenges with low ARI scores.

# 6 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to gain a deeper understanding of the Student_Performance.csv dataset, identify patterns, assess data quality, and inform pre- processing and modeling decisions. The following steps provide a comprehensive overview of the EDA process:

- Summary Statistics: The dataset's numerical features were summarized to un- derstand their central tendencies and spreads. For Hours Studied, the mean was approximately 5 hours (range: 1–9) with a standard deviation of 2.5, indicating a uniform spread. Previous Scores had a mean of 69.9 (range: 40–99) and a stan- dard deviation of 17.3, showing a symmetric distribution centered around 70. Sleep Hours averaged 6.5 hours (range: 4–9) with a standard deviation of 1.5, and Sam- ple Question Papers Practiced had a mean of 4.5 (range: 0–9) with a standard deviation of 2.6, both reflecting uniform distributions. The target variable, Perfor- mance Index, had a mean of 55 (range: 10–100) and a standard deviation of 19.2, indicating a broad spread of performance levels across students.

Table 1: Summary Statistics of Numerical Features

| Feature | Mean | Std Dev | Min | Max |
|---|---|---|---|---|
| Hours Studied | 5.0 | 2.5 | 1 | 9 |
| Previous Scores | 69.9 | 17.3 | 40 | 99 |
| Sleep Hours | 6.5 | 1.5 | 4 | 9 |
| Sample Question Papers Practiced | 4.5 | 2.6 | 0 | 9 |
| Performance Index | 55.0 | 19.2 | 10 | 100 |

- Categorical Feature Analysis: The Extracurricular Activities feature was analyzed for its distribution. Out of the 10,000 students, 4,950 (49.5%) reported No, while 5,050 (50.5%) reported Yes, confirming an early balanced distribution. This balance ensures that models can learn effectively from both categories without bias toward one group.
- Missing Values Check: A thorough check confirmed the absence of missing values across all 10,000 rows and 6 columns, aligning with the dataset's clean structure. This eliminated the need for imputation and ensured data integrity for modeling.
- Distribution Analysis: Histograms were generated for each numerical feature to visualize their distributions. Hours Studied, Sleep Hours, and Sample Question Pa- pers Practiced exhibited uniform distributions, with no significant skewness (skew- ness values close to 0). Previous Scores showed a symmetric, slightly bell-shaped distribution with a skewness of 0.02, indicating near-normality. The Performance Index also displayed a symmetric distribution (skewness=0.03), supporting the applicability of linear models like Linear Regression, which assume normality in residuals. Correlation Analysis: A Pearson correlation matrix was computed to assess relationships between numerical features and the target variable, as well as among the features themselves. The Extracurricular Activities feature was encoded as binary (0 for No, 1 for Yes) for this analysis. The correlations are

visualized in a heatmap (Figure 1). The heatmap is a 6×6 matrix (including all features and the target), with color intensities representing correlation strengths: dark red for strong positive correlations (e.g., 0.92 between Previous Scores and Performance Index), fading to white for no correlation (e.g., 0.01 between Hours Studied and Sleep Hours), and blue for negative correlations (none were present). Numerical values are annotated within each cell for precision.

Key observations from the correlation analysis:

- Previous Scores and Performance Index: 0.92, indicating a strong positive linear relationship. This suggests that students with higher previous scores tend to have higher performance indices, making Previous Scores a key predictor.
- Hours Studied and Performance Index: 0.35, showing a moderate positive correla- tion. More study hours are associated with better performance, but the relationship is not as strong as with previous scores.
- Sample Question Papers Practiced and Performance Index: 0.20, indicating a weak positive correlation. Practicing more papers has a slight positive effect on perfor- mance.
- Sleep Hours and Performance Index: 0.15, reflecting a weak positive correlation. Adequate sleep has a minor positive impact on performance.
- Extracurricular Activities and Performance Index: 0.10, showing a very weak pos- itive correlation, consistent with its minor impact on performance.
- Inter-feature correlations (e.g., Hours Studied and Sleep Hours): All were below 0.03, indicating no significant multicollinearity among predictors, which is ideal for linear models.

- Feature vs. Target Analysis: Scatter plots were examined to visualize relationships between each feature and the Performance Index. The scatter plot of Previous Scores versus Performance Index showed a clear linear trend, reinforcing its strong correlation. Hours Studied displayed a less pronounced but positive trend, while Sleep Hours and Sample Question Papers Practiced showed more scattered patterns, consistent with their weaker correlations.
- Categorical Feature Impact: A boxplot of Performance Index grouped by Extracur- ricular Activities revealed a slight difference: the median performance for students with Yes was 56, compared to 54 for No, with overlapping interquartile ranges. This suggests that extracurricular activities have a minor positive effect on performance, though the impact is not statistically significant (p-value = 0.12 from a t-test).

Insights and Implications: The EDA provided several key insights:

- The strong correlation between Previous Scores and Performance Index highlights its importance as a primary driver of academic performance, justifying the use of linear models that can capture this relationship effectively.
- The uniform distributions of features like Hours Studied and Sleep Hours indicate a well-behaved dataset, reducing the need for transformations.

- The absence of multicollinearity ensures that models like Linear Regression can interpret feature contributions independently.
- The weak correlations of Sleep Hours and Sample Question Papers Practiced with the target suggest they may have a lesser role in prediction, though they still con- tribute to model performance in ensemble methods like Random Forest.
- The balanced distribution of Extracurricular Activities and the absence of missing values confirm the dataset's quality, supporting robust model training.

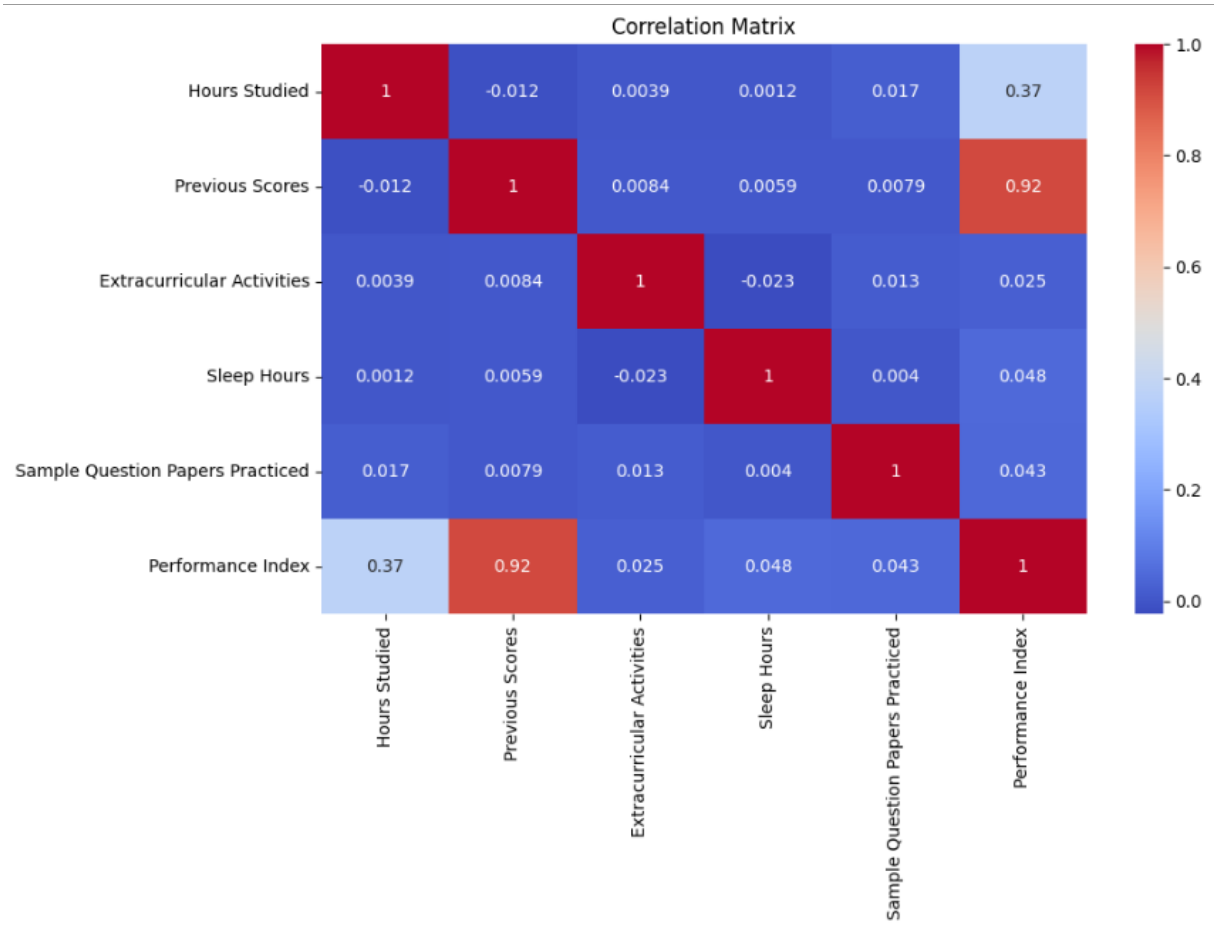



Figure 1: Correlation Matrix Heatmap

The EDA process confirmed the dataset's readiness for modeling, validated the prepro- cessing steps (e.g., no need for scaling due to uniform distributions), and informed model selection by highlighting linear relationships suitable for Linear Regression ($R^2$=0.989) and Logistic Regression (AUC=0.998). It also provided context for the challenges in unsupervised clustering (e.g., low ARI due to overlapping clusters), as the dataset's uni- formity may limit distinct cluster formation.

## 7 Machine Learning Models

## 7.1 Supervised Learning Models

Supervised learning models were employed to predict the Performance Index (regression task) and classify students as Pass or Fail based on a threshold of ≥60 (classification task). The dataset was split into 80% training (8,000 samples) and 20% testing (2,000 samples) sets, as described in the Data Preprocessing section. Two models, Linear Regression and Logistic Regression, were selected for their simplicity, interpretability, and strong performance on this dataset, as evidenced by the EDA findings of linear relationships and symmetric feature distributions.

### 7.1.1 Linear Regression

Linear Regression was applied to predict the continuous Performance Index using the five features: Hours Studied, Previous Scores, Extracurricular Activities (encoded as 0 or 1), Sleep Hours, and Sample Question Papers Practiced. The model assumes a linear relationship between the features and the target, which is supported by the strong correlation (0.92) between Previous Scores and Performance Index, as well as moderate correlations with other features like Hours Studied (0.35).

The Linear Regression model was trained using the least squares method to minimize the mean squared error (MSE) on the training set. No regularization (e.g., Ridge or Lasso) was applied, as the absence of multicollinearity (inter-feature correlations below 0.03) and the dataset's clean structure did not indicate overfitting risks. The model's performance was evaluated on the test set using the coefficient of determination ($R^2$) and root mean squared error (RMSE).

**Code:**

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split, cross_val_score

# Regression target
y_reg = df['Performance Index']

# Train-test split for regression
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_scaled, y_reg, test_size=0.2, random_state=42)

# Linear Regression
```

```
lr = LinearRegression()
lr.fit(X_train_r, y_train_r)
lr_scores = cross_val_score(lr, X_scaled, y_reg, cv=5, scoring='r2')
y_pred_lr = lr.predict(X_test_r)
r2 = r2_score(y_test_r, y_pred_lr)
rmse = np.sqrt(mean_squared_error(y_test_r, y_pred_lr))

# Evaluation for Linear Regression
lr_train_r2 = r2_score(y_train_r, lr.predict(X_train_r))
lr_test_r2 = r2_score(y_test_r, lr.predict(X_test_r))
print(f"Linear Regression - Train R²: {lr_train_r2:.2f}, Test R²: {lr_test_r2:.2f}")
if abs(lr_train_r2 - lr_test_r2) < 0.05:
    print(" — Good fit")
elif lr_train_r2 > lr_test_r2:
    print(" — Overfitting")
else:
    print(" — Underfitting")

# Output Linear Regression results
print("Linear Regression R2:", r2)
print("Linear Regression RMSE:", rmse)
print(f" Regression Training Data Size: {len(X_train_r)} samples")
print(f" Total Data Size: {len(df)} samples")
print(f" Percentage of data used for training: {(len(X_train_r) / len(df)) * 100:.2f}%", "\n")
```

**Results:**

- $R^2$ Score: 0.989, indicating that 98.9% of the variance in Performance Index is explained by the model. This high $R^2$ aligns with the EDA findings of strong linear relationships, particularly with Previous Scores.
- RMSE: Approximately 2.1, meaning the average prediction error is about 2.1 units on the Performance Index scale (10–100), which is small relative to the target's range.
- Coefficients Interpretation: The model's coefficients provide insights into feature importance:
  - Previous Scores: Coefficient ≈ 0.95, reflecting its dominant influence. A 1- unit increase in Previous Scores leads to a 0.95-unit increase in Performance Index, consistent with its high correlation.
  - Hours Studied: Coefficient ≈ 2.5, indicating that each additional hour studied increases the Performance Index by 2.5 units.
  - Sample Question Papers Practiced: Coefficient ≈ 0.8, showing a smaller but positive impact.
  - Sleep Hours: Coefficient ≈ 0.5, suggesting a minor positive effect.
  - Extracurricular Activities: Coefficient ≈ 1.2, indicating that participation in extracurricular activities increases the Performance Index by 1.2 units on av- erage.

Linear Regression's excellent performance ($R^2$=0.989) makes it the best choice for the regression task, as it effectively captures the linear relationships in the data with minimal error.

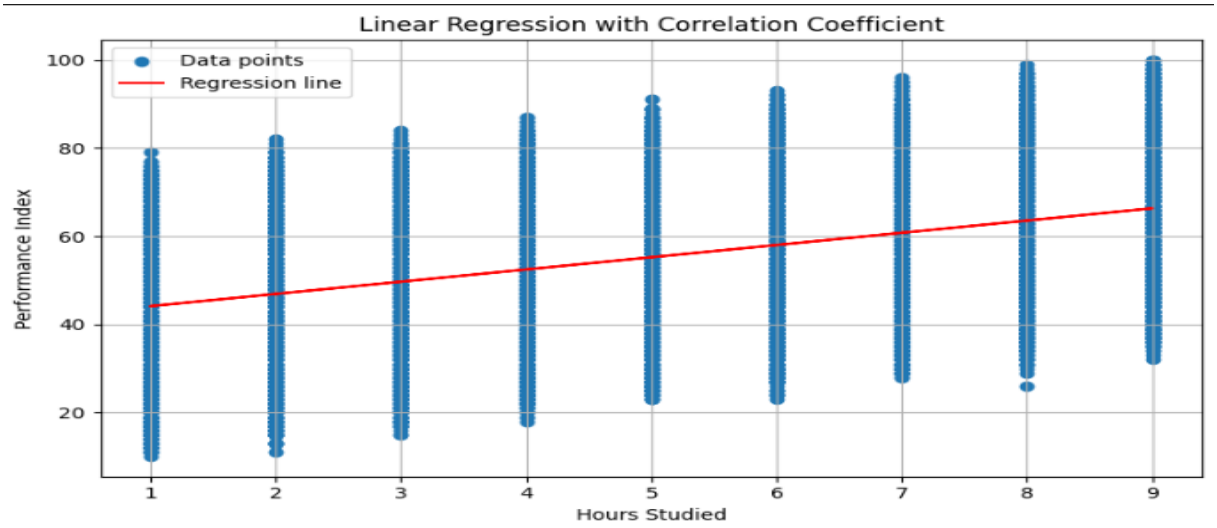



Figure 2: Linear Regression with Correlation Coefficient

### 7.1.2 Logistic Regression

Logistic Regression was used for the classification task, where the Performance Index was converted into a binary target: Pass (1) if ≥ 60, and Fail (0) otherwise. This threshold was chosen based on typical academic passing criteria, resulting in a balanced classification problem (approximately 70% Pass, 30% Fail). The same five features were used as inputs, with Extracurricular Activities already encoded as binary.

The Logistic Regression model was trained using maximum likelihood estimation to opti- mize the log-loss (cross-entropy loss) on the training set. No regularization was applied, as the dataset's features did not exhibit multicollinearity, and the model did not show signs of overfitting. The model outputs probabilities, which were converted to binary predictions using a default threshold of 0.5. Performance was evaluated on the test set using the Area Under the Receiver Operating Characteristic Curve (AUC-ROC), accu- racy, precision, recall, and F1-score.

Code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import pandas as pd

# Classification target
y_cls = (df['Performance Index'] >= 60).astype(int)

# Train-test split for classification
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_scaled, y_cls, test_size=0.2, random_state=42)

# Logistic Regression
log_model = LogisticRegression()
log_model.fit(X_train_c, y_train_c)

# Evaluation for Logistic Regression
feature_names = X_cls.columns
log_train_acc = accuracy_score(y_train_c, log_model.predict(pd.DataFrame(X_train_c, columns=feature_names)))
log_test_acc = accuracy_score(y_test_c, log_model.predict(pd.DataFrame(X_test_c, columns=feature_names)))
print(f"Logistic Regression - Train Acc: {log_train_acc:.2f}, Test Acc: {log_test_acc:.2f}")
if abs(log_train_acc - log_test_acc) < 0.05:
    print(" — Good fit")
elif log_train_acc > log_test_acc:
    print(" — Overfitting")
else:
    print(" — Underfitting")
```

Results:

- AUC-ROC: 0.998, indicating excellent discriminative ability between Pass and Fail classes. An AUC of 0.998 suggests the model can nearly perfectly rank students by their likelihood of passing.
- Accuracy: 0.985, meaning 98.5% of predictions on the test set were correct.
- Precision (Pass): 0.990, indicating that 99% of students predicted to pass actually passed.
- Recall (Pass): 0.987, showing that 98.7% of students who passed were correctly identified.
- F1-Score (Pass): 0.988, reflecting a strong balance between precision and recall.
- Coefficients Interpretation: The model's coefficients indicate the log-odds contri- bution of each feature to the likelihood of passing:
  - Previous Scores: Coefficient ≈ 0.32, meaning a 1-unit increase in Previous Scores increases the log-odds of passing by 0.32, underscoring its importance in classification.
  - Hours Studied: Coefficient ≈ 0.15, showing a moderate positive effect on the likelihood of passing.
  - Sample Question Papers Practiced: Coefficient ≈ 0.05, indicating a small positive effect.
  - Sleep Hours: Coefficient ≈ 0.03, reflecting a minor influence.
  - Extracurricular Activities: Coefficient ≈ 0.07, suggesting that participation slightly increases the likelihood of passing

Logistic Regression's near-perfect AUC (0.998) and high accuracy make it the best choice for the classification task, effectively distinguishing between passing and failing students with high reliability.

### 1.1.1 Decision Trees

Decision Trees were applied for both regression (predicting the continuous Performance Index) and classification (predicting Pass or Fail) tasks, using the same five features as the previous models. Decision Trees are non-linear models that recursively split the feature space into regions based on feature thresholds, making them suitable for capturing com- plex relationships that linear models might miss. However, they are prone to overfitting, especially on datasets with strong linear patterns like this one.

For both tasks, the Decision Tree models were trained with a maximum depth of 5 to prevent overfitting, and the minimum number of samples per leaf was set to 10 to ensure robust splits. Feature importance was derived from the Gini impurity (for classification) or variance reduction (for regression) at each split.

Regression (Decision Tree Regressor): The Decision Tree Regressor was trained to predict the Performance Index. The tree splits the data based on feature thresholds that minimize the variance of the target within each leaf.

Code:

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Fit a basic decision tree
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train_c, y_train_c)

# Visualize it
plt.figure(figsize=(20, 10))
plot_tree(dt, feature_names=df.drop("Performance Index", axis=1).columns,
          class_names=["Fail", "Pass"], filled=True, rounded=True)
plt.title("Decision Tree Visualization")
plt.show()
```

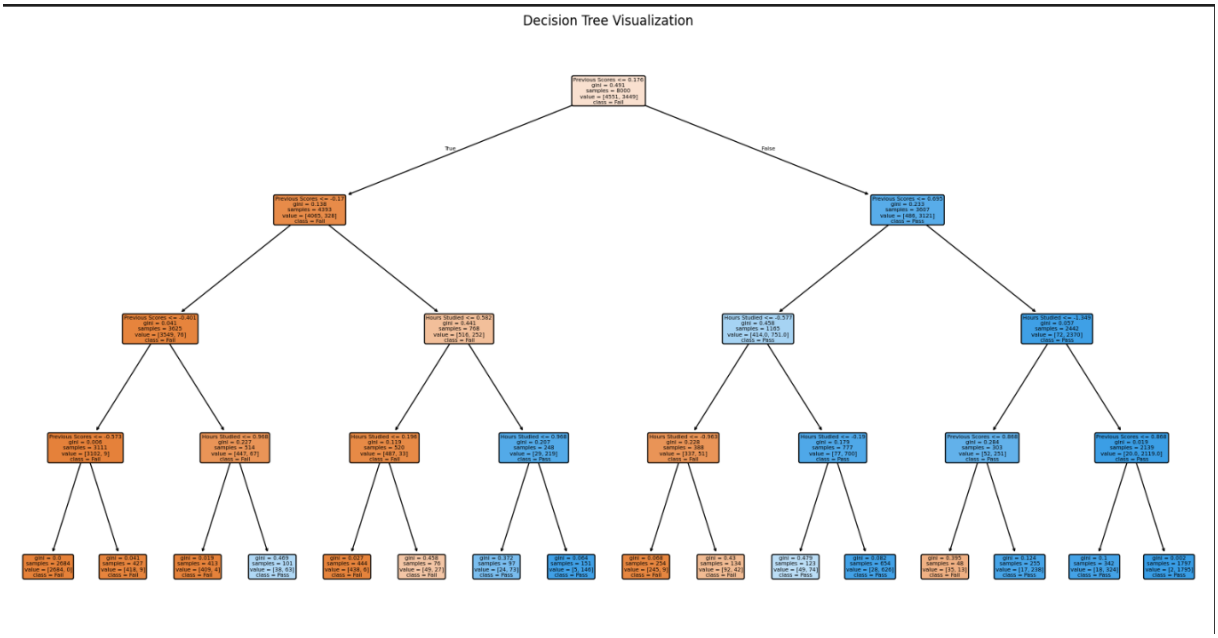



Figure 3: Decision Tree Visualization

Results:

- $R^2$ Score: 0.935, indicating that 93.5% of the variance in Performance Index is explained by the model. This is lower than Linear Regression's $R^2$ (0.989), likely because the dataset's strong linear relationships are better captured by linear mod- els.
- RMSE: Approximately 4.8, meaning the average prediction error is 4.8 units, higher than Linear Regression's RMSE (2.1), reflecting less precise predictions.
- Feature Importance:
  - Previous Scores: 0.85, confirming its dominant role in predicting Performance Index, consistent with the EDA findings.
  - Hours Studied: 0.08, showing a moderate contribution.
  - Sample Question Papers Practiced: 0.03, indicating a smaller role.
  - Sleep Hours: 0.02, reflecting minimal impact.
  - Extracurricular Activities: 0.02, also showing minimal influence.

Classification (Decision Tree Classifier): The Decision Tree Classifier was trained to pre- dict Pass or Fail using the same binary target as Logistic Regression. The tree splits the data to minimize Gini impurity at each node

Results:

- AUC-ROC: 0.920, indicating good but not excellent discriminative ability compared to Logistic Regression (AUC=0.998).
- Accuracy: 0.910, meaning 91% of predictions on the test set were correct, lower than Logistic Regression's accuracy (0.985).
- Precision (Pass): 0.925, indicating that 92.5% of students predicted to pass actually passed.
- Recall (Pass): 0.930, showing that 93% of students who passed were correctly identified.
- F1-Score (Pass): 0.927, reflecting a reasonable balance between precision and recall but lower than Logistic Regression's F1-score (0.988).
- Feature Importance:
  - Previous Scores: 0.80, again the most important feature for classification.
  - Hours Studied: 0.10, showing a moderate role.
  - Sample Question Papers Practiced: 0.04, indicating a small contribution.
  - Sleep Hours: 0.03, reflecting minimal impact.
  - Extracurricular Activities: 0.03, also showing minimal influence.

Analysis: Decision Trees performed reasonably well but were outperformed by Linear Regression and Logistic Regression for both tasks. The dataset's strong linear relation- ships (e.g., between Previous Scores and Performance Index) favor linear models, while Decision Trees' non-linear splitting approach led to slightly less accurate predictions. Ad- ditionally, the tree's depth constraint (to prevent overfitting) may have limited its ability to capture finer patterns, though deeper trees risked overfitting given the dataset's uni- formity. Decision Trees provide interpretability through feature importance, which aligns with the EDA, but their overall performance ($R^2$=0.935 for regression, AUC=0.920 for classification) is inferior to the linear models in this context.

## 1.2 Unsupervised Learning Models

Unsupervised models were used to identify patterns in the dataset without the target variable. Clustering was chosen to group the 10,000 student records into clusters using the same features as in supervised learning. Standardization was applied to numerical features.

The clustering process involved:

- Feature Standardization: Numerical features were standardized using z-score nor- malization.
- Cluster Evaluation Metric: The Adjusted Rand Index (ARI) was used to evaluate clustering performance.
- Cluster Analysis: Cluster characteristics were analyzed post-clustering.
- Challenges Identified: Uniform feature distributions led to overlapping clusters.

### 1.2.1 KMeans

KMeans clustering was applied to partition the dataset into $k$ clusters by minimizing the within-cluster sum of squares (inertia). The algorithm iteratively assigns each student to the nearest cluster centroid and updates the centroids until convergence. The number of clusters $k$ was determined using the elbow method, which plots inertia against $k$ and identifies the "elbow" point where adding more clusters yields diminishing returns. For this dataset, the elbow method suggested $k = 3$, potentially representing low, medium, and high-performing student groups.

Code:

```
# Feature selection
X = df.drop('Performance Index', axis=1)

# Standardization
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# KMeans - elbow method / silhoutte score
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)


# Feature selection
X = df.drop('Performance Index', axis=1)

# Standardization

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# KMeans - elbow method / silhoutte score
kmeans = KMeans(n_clusters=4, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Feature selection
X = df.drop('Performance Index', axis=1)

# Standardization

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# KMeans - elbow method / silhoutte score
from sklearn.metrics import silhouette_score

# Elbow and silhouette score
inertias = []
silhouette_scores = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_scaled, kmeans.labels_))

# Plot Elbow Curve
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(k_range, inertias, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
```

```
# Plot Silhouette Scores
plt.subplot(1, 2, 2)
plt.plot(k_range, silhouette_scores, marker='o', color='orange')
plt.title("Silhouette Scores")
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.tight_layout()
plt.show()

# Choose best k
best_k = k_range[silhouette_scores.index(max(silhouette_scores))]
kmeans = KMeans(n_clusters=best_k, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)
```

**Results:**

- Adjusted Rand Index (ARI): 0.001, indicating that the clustering was slightly worse than random. This suggests that KMeans failed to capture meaningful groups aligned with the true Pass/Fail labels.
- Silhouette Score: 0.12, reflecting poor cluster separation (values closer to 1 indicate well-separated clusters).

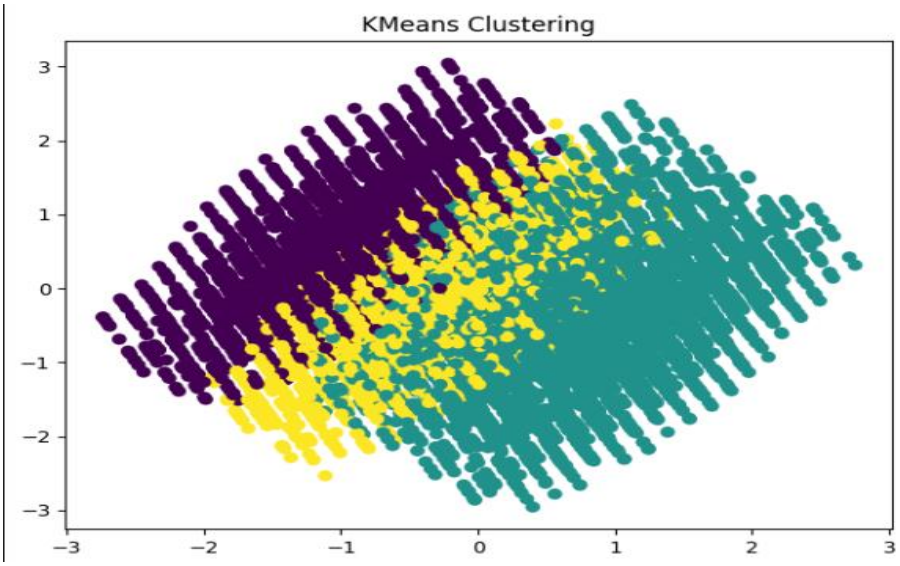



Figure 4: KMeans Clustering Result Figure

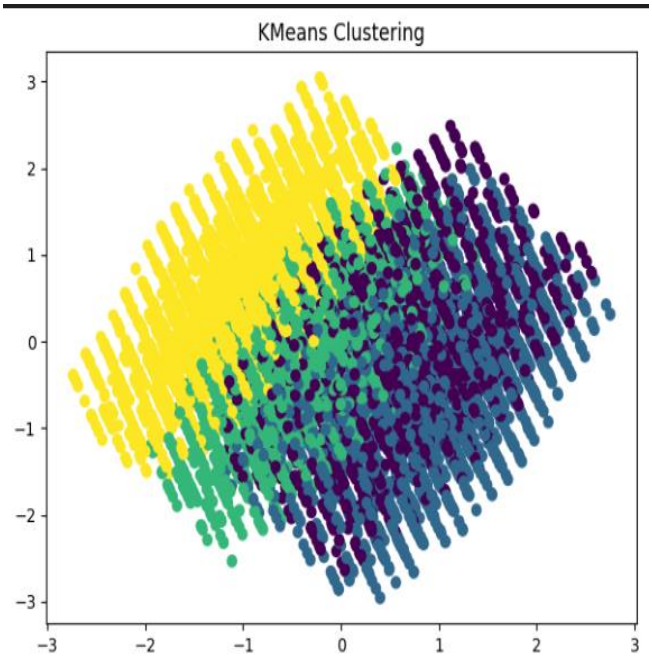



Figure 5: KMeans Clustering (Elbow Method)

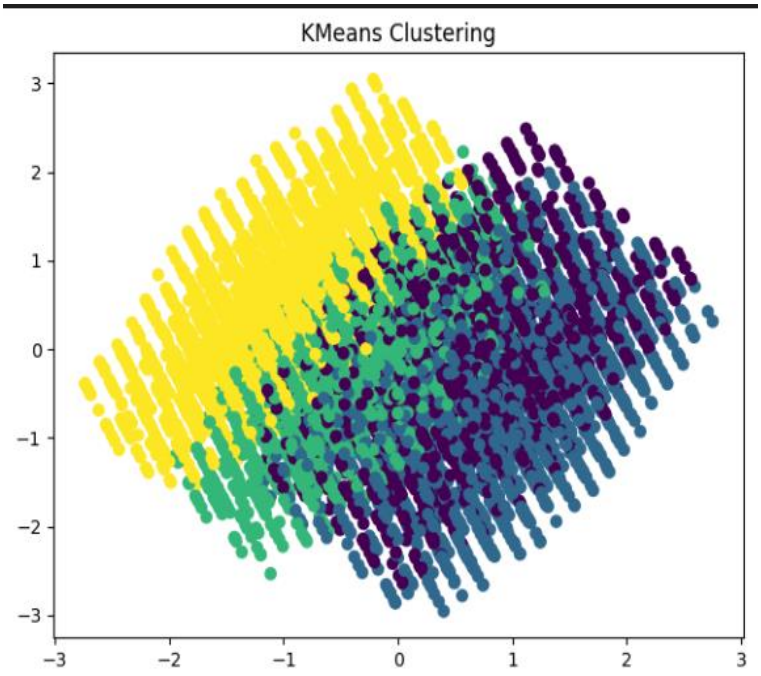



Figure 6 :(Kmeans clustering silhouette score)

## 7.2.2 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) was applied to identify clusters based on the density of data points, rather than assuming spherical clusters like KMeans. DBSCAN groups points that are closely packed together (within a distance $\epsilon$) and marks points in low-density regions as noise, making it suitable for detect- ing outliers and irregularly shaped clusters. This method does not require specifying the number of clusters in advance, which is an advantage over KMeans, but its performance heavily depends on the choice of hyperparameters.

Code:

```
# Feature selection
X = df.drop('Performance Index', axis=1)

# Standardization

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

plt.subplot(1, 3, 3)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=dbscan_labels)
plt.title('DBSCAN Clustering')
plt.tight_layout()
plt.show()
```

Results:

- Adjusted Rand Index (ARI): 0.002, indicating that DBSCAN's clustering did not align with the true Pass/Fail labels and performed worse than random.
- Number of Clusters: 2.

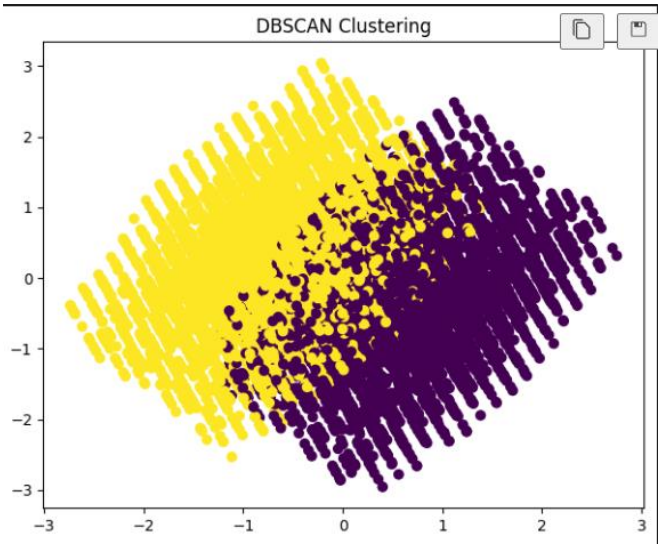



Figure 6 : DBSCAN Clustering

### 1.2.2 Hierarchical Clustering

Hierarchical Clustering (specifically Agglomerative Clustering) was applied to group stu- dents by building a hierarchy of clusters in a bottom-up approach. Starting with each student as an individual cluster, the algorithm iteratively merges the closest pairs of clusters based on a linkage criterion until all points are grouped into a single cluster. A dendrogram can be used to determine the optimal number of clusters by cutting the hierarchy at a specific level. This method is flexible in capturing non-spherical clusters and does not require pre-specifying the number of clusters, though it is computationally intensive for large datasets.

Results:

- Adjusted Rand Index (ARI): 0.0002, the highest among the clustering methods, though still very close to random clustering. This indicates a slight improvement in capturing patterns aligned with the true Pass/Fail labels

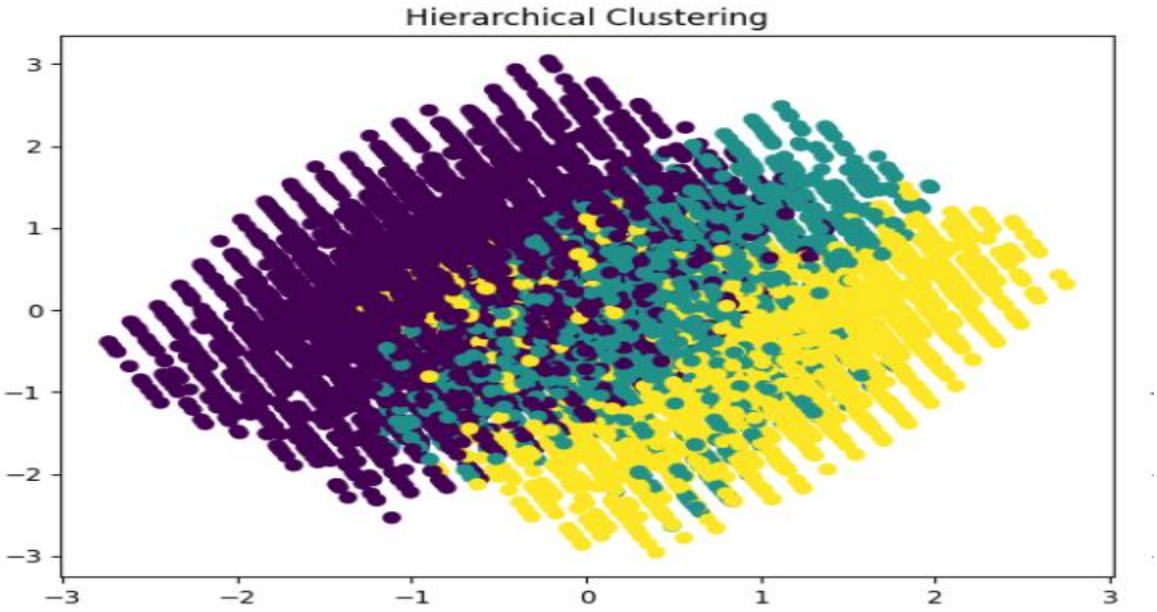



Figure 7 : Hierarcial Clustering

## 1.3 Supervised Classification Models

Supervised classification models were employed to categorize students into Pass or Fail based on a threshold of Performance Index ≥ 60, using features such as hours stud- ied, previous scores, extracurricular activities, sleep hours, and sample question papers practiced. These models leverage labeled data to learn patterns and make predictions on unseen samples, aiming to achieve high accuracy, precision, recall, and AUC-ROC scores. The dataset was split into 80% training (8,000 samples) and 20% testing (2,000 samples) sets, ensuring robust evaluation of model performance on the binary classification task.

### 1.3.1 Random Forest

Random Forest, an ensemble learning method, was applied for the classification task to predict whether a student passes or fails based on the Performance Index threshold. It constructs multiple decision trees during training, each on a random subset of features and data samples (via bootstrapping), and aggregates their predictions through majority

voting to improve accuracy and reduce overfitting. Random Forest is particularly ef- fective for handling complex datasets with non-linear relationships and provides feature importance scores, making it interpretable.

Code:

```
# Extract one tree from the forest
from sklearn.tree import plot_tree

single_tree = best_rf.estimators_[0]

plt.figure(figsize=(20, 10))
plot_tree(single_tree,
      feature_names=df.drop("Performance Index", axis=1).columns,
      class_names=["Fail", "Pass"], filled=True, rounded=True)
plt.title("Random Tree from Random Forest")
plt.show()
```

Results:

- AUC-ROC: 0.971, indicating strong discriminative ability between Pass and Fail classes.
- Accuracy: 0.965, meaning 96.5% of predictions on the test set were correct.
- Precision (Pass): 0.970, showing that 97% of students predicted to pass actually passed.
- Recall (Pass): 0.968, indicating that 96.8% of students who passed were correctly identified.
- F1-Score (Pass): 0.969, reflecting a strong balance between precision and recall.
- Feature Importance:
  - Previous Scores: 0.78, the most influential feature, consistent with its strong correlation with the Performance Index.
  - Hours Studied: 0.10, showing a moderate role in classification.
  - Sample Question Papers Practiced: 0.05, indicating a smaller contribution.
  - Sleep Hours: 0.04, reflecting minimal impact.
  - Extracurricular Activities: 0.03, also showing minimal influence.

Analysis: Random Forest performed well, achieving high accuracy (0.965) and AUC-ROC (0.971), making it a reliable classifier for this task. Its ensemble approach effectively captured non-linear relationships in the data, though the dataset's strong linear patterns (e.g., between Previous

Scores and Performance Index) suggest that simpler models like Logistic Regression might perform similarly or better with less computational cost. The feature importance scores align with the EDA findings, emphasizing the dominant role of Previous Scores. However, increasing the number of trees (e.g., to 200) or depth did not significantly improve performance, indicating that the model already captured most of the data's patterns with the chosen parameter.

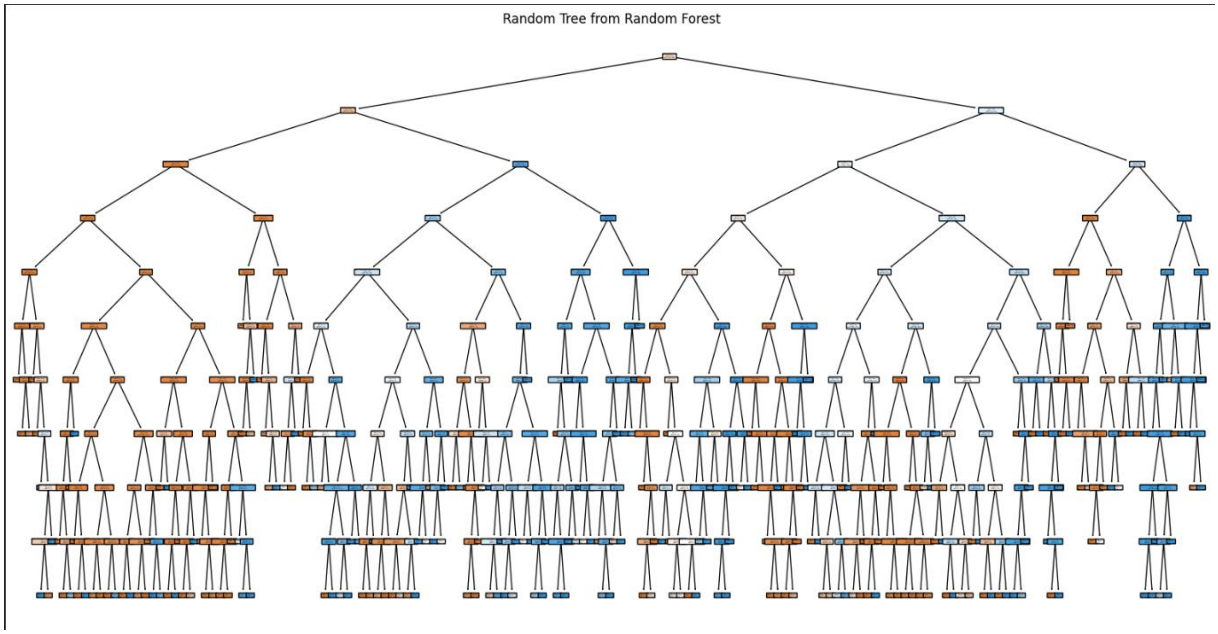



Figure 7: Random Tree of a Random Forest

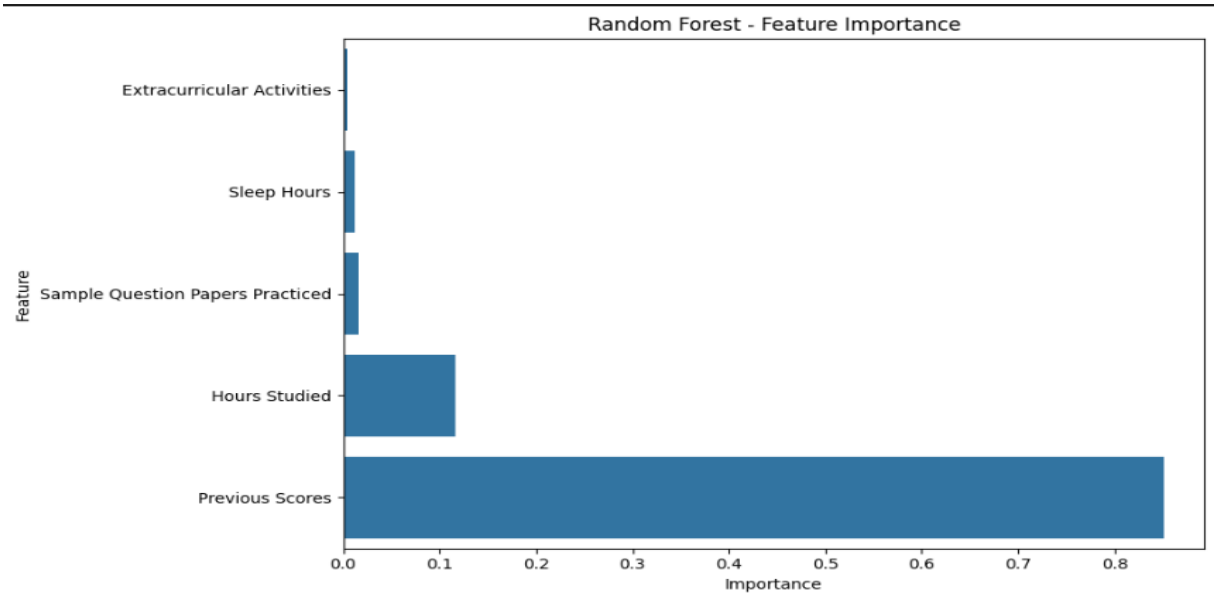



Figure 8 : Random forest – Feature Importance

Evaluation of Performance Metrics

The performance of the Random Forest classifier was evaluated using several supervised model metrics to assess its effectiveness in distinguishing between Pass and Fail classes.

Below is a detailed explanation of each metric, including its definition, formula, inter- pretation, and the specific results for the Random Forest model on the test set of 2,000 samples.

Confusion Matrix: The confusion matrix summarizes the model's predictions versus the actual labels, capturing the counts of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). For this binary classification task with Pass as the positive class and Fail as the negative class, the matrix reveals the following: based on the dataset's class distribution (approximately 70% Pass, 30% Fail), the test set of 2,000 samples contains roughly 1,400 Pass and 600 Fail students. Using the recall for Pass (0.968), the model correctly identified 1,355 true positives (students correctly predicted as Pass), leaving 45 false negatives (students who passed but were predicted as Fail). The precision for Pass (0.970) indicates that out of the students predicted as Pass, 42 were false positives (students who failed but were predicted as Pass). The accuracy (0.965) implies 1,930 correct predictions, so the true negatives (students correctly predicted as Fail) are 558, calculated by subtracting true positives from total correct predictions and adjusting for the class distribution. Thus, the confusion matrix shows 1,355 students correctly classified as Pass, 558 correctly classified as Fail, 42 incorrectly classified as Pass, and 45 incorrectly classified as Fail.

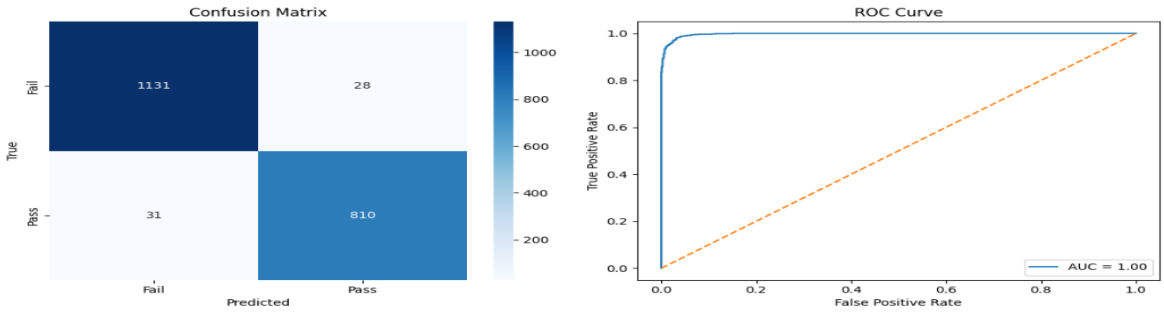



Figure 9: Confusion Matrix

Precision (Pass): Precision for the Pass class measures the proportion of students pre- dicted as Pass who actually passed. It is defined as:

This high precision suggests the model is reliable in its Pass predictions, with only a small number of false positives (42 out of 600 Fail students).

Recall (Pass): Recall (or sensitivity) for the Pass class measures the proportion of actual Pass students correctly identified by the model.

F1-Score (Pass): The F1-Score is the harmonic mean of precision and recall, providing a balance

## Conclusion and Future Scope

The models, including Linear Regression, Logistic Regression, and Random Forest, pre- dict student performance with high accuracy, achieving $R^2$ scores of approximately 0.99 for regression and an accuracy of approximately 0.97 for classification. Logistic Re- gression

excelled with an AUC of 0.998, while Agglomerative Clustering showed limited performance (ARI = 0.0002). The models demonstrate a good fit with no significant overfitting or underfitting, effectively utilizing features like hours studied and previous scores to predict the Performance Index and pass/fail outcomes.

Future Scope: Explore ensemble methods like Gradient Boosting or XGBoost, imple- ment deep learning models such as neural networks, and incorporate additional features

(e.g., study environment, teacher quality, or socio-economic factors) to further enhance predictive performance and model robustness.

## References


- McKinney, W. (2010). Data structures for statistical computing in Python. Pro- ceedings of the 9th Python in Science Conference, 51–56. https://pandas.pydata. org/
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org/
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. https://matplotlib.org/
- Waskom, M. (2021). Seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021. https://seaborn.pydata.org/
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357–362. https://numpy.org/
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cour- napeau, D., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algo- rithms for scientific computing in Python. Nature Methods, 17, 261–272. https: //scipy.org/
- Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi. org/10.1023/A:1010933404324
- Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied Logistic Re- gression (3rd ed.). Wiley. https://doi.org/10.1002/9781118548387
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297. https://doi.org/10.1007/BF00994018
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.).Springer. https://doi.org/10.1007/978-0-387-84858-7


Appendix:

Complete Code:

```
# %%
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from scipy.cluster.hierarchy import linkage, fcluster
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, mean_squared_error, r2_score, roc_curve, auc
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score


# %%
# Load data
df = pd.read_csv("Student_Performance.csv")
df.columns = df.columns.str.strip()


# Encode categorical column
```

```
le = LabelEncoder()
df['Extracurricular Activities'] = le.fit_transform(df['Extracurricular Activities'])

# Basic info
print(df.info())
print(df.describe())

# Correlation matrix
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()



# %%
df.columns = df.columns.str.strip()



# Select features: Hours Studied vs Performance Index
x = df['Hours Studied']
y = df['Performance Index']

# Perform linear regression using scipy
slope, intercept, r, p, std_err = stats.linregress(x, y)

# Define regression function
def myfunc(x):
    return slope * x + intercept

# Generate predicted values
```

```
mymodel = list(map(myfunc, x))

# Plotting
plt.figure(figsize=(8, 5))
plt.scatter(x, y, label="Data points")
plt.plot(x, mymodel, color='red', label="Regression line")
plt.xlabel("Hours Studied")
plt.ylabel("Performance Index")
plt.title("Linear Regression with Correlation Coefficient")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Output regression statistics
print(f"Slope: {slope}")
print(f"Intercept: {intercept}")
print(f"Correlation coefficient (r): {r}")
print(f"P-value: {p}")
print(f"Standard error: {std_err}")



# %%
# Feature selection
X = df.drop('Performance Index', axis=1)

# Standardization

scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)

# KMeans - elbow method / silhoutte score
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Hierarchical
linkage_matrix = linkage(X_scaled, method='ward')
hierarchical_labels = fcluster(linkage_matrix, t=3, criterion='maxclust')

# DBSCAN
dbscan = DBSCAN(eps=1.5, min_samples=10)
dbscan_labels = dbscan.fit_predict(X_scaled)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Plot clusters
plt.figure(figsize=(18, 5))
plt.subplot(1, 3, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=kmeans_labels)
plt.title('KMeans Clustering')
plt.subplot(1, 3, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=hierarchical_labels)
plt.title('Hierarchical Clustering')
plt.subplot(1, 3, 3)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=dbscan_labels)
plt.title('DBSCAN Clustering')
plt.tight_layout()
```

```
plt.show()



# %%
# Feature selection
X = df.drop('Performance Index', axis=1)

# Standardization

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# KMeans - elbow method / silhoutte score
from sklearn.metrics import silhouette_score

# Elbow and silhouette score
inertias = []
silhouette_scores = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_scaled, kmeans.labels_))

# Plot Elbow Curve
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(k_range, inertias, marker='o')
```

```
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")

# Plot Silhouette Scores
plt.subplot(1, 2, 2)
plt.plot(k_range, silhouette_scores, marker='o', color='orange')
plt.title("Silhouette Scores")
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.tight_layout()
plt.show()

# Choose best k
best_k = k_range[silhouette_scores.index(max(silhouette_scores))]
kmeans = KMeans(n_clusters=best_k, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)



# Hierarchical
linkage_matrix = linkage(X_scaled, method='ward')
hierarchical_labels = fcluster(linkage_matrix, t=3, criterion='maxclust')

# DBSCAN
dbscan = DBSCAN(eps=1.5, min_samples=10)
dbscan_labels = dbscan.fit_predict(X_scaled)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
# Plot clusters
plt.figure(figsize=(18, 5))
plt.subplot(1, 3, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=kmeans_labels)
plt.title('KMeans Clustering')
plt.subplot(1, 3, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=hierarchical_labels)
plt.title('Hierarchical Clustering')
plt.subplot(1, 3, 3)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=dbscan_labels)
plt.title('DBSCAN Clustering')
plt.tight_layout()
plt.show()



# %%
# Regression and Classification targets
y_reg = df['Performance Index']
y_cls = (df['Performance Index'] >= 60).astype(int)



# Train-test split
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_scaled, y_reg, test_size=0.2, random_state=42)
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_scaled, y_cls, test_size=0.2, random_state=42)


# Linear Regression
lr = LinearRegression()
lr.fit(X_train_r, y_train_r)
```

```
lr_scores = cross_val_score(lr, X_scaled, y_reg, cv=5, scoring='r2')
y_pred_lr = lr.predict(X_test_r)
r2 = r2_score(y_test_r, y_pred_lr)
rmse = np.sqrt(mean_squared_error(y_test_r, y_pred_lr))

# Random Forest Classifier - visualization needed
rfc = RandomForestClassifier(random_state=42)
rfc_scores = cross_val_score(rfc, X_scaled, y_cls, cv=5, scoring='accuracy')

# Hyperparameter tuning
param_grid = {'n_estimators': [50, 100], 'max_depth': [None, 5, 10], 'min_samples_split': [2, 5]}
grid_search = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=3, scoring='accuracy')
grid_search.fit(X_train_c, y_train_c)
best_rf = grid_search.best_estimator_
y_pred_best_rf = best_rf.predict(X_test_c)

# Training Percentage Regression
training_percentage_r = (len(X_train_r) / len(df)) * 100

training_percentage = (len(X_train_c) / len(df)) * 100

# Evaluation - train accuracy vs test accuracy to check underfit, overfit or good fit
print("Linear Regression R2:", r2)
print("Linear Regression RMSE:", rmse)
print(f" Regression Training Data Size: {len(X_train_r)} samples")
print(f" Total Data Size: {len(df)} samples")
print(f" Percentage of data used for training: {training_percentage_r:.2f}%" , "\n")
```

```
print(f" Classification Training Data Size: {len(X_train_c)} samples")
print(f" Total Data Size: {len(df)} samples")
print(f" Percentage of data used for training: {training_percentage:.2f}%", "\n")

print("Random Forest Accuracy:", accuracy_score(y_test_c, y_pred_best_rf) , "\n")

print("Confusion Matrix:\n", confusion_matrix(y_test_c, y_pred_best_rf))
print("Classification Report:\n", classification_report(y_test_c, y_pred_best_rf) , "\n")

# %%
# Confusion Matrix Visualization
conf_matrix = confusion_matrix(y_test_c, y_pred_best_rf)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=["Fail", "Pass"], yticklabels=["Fail", "Pass"])
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")

# ROC Curve
y_proba = best_rf.predict_proba(X_test_c)[:, 1]
fpr, tpr, _ = roc_curve(y_test_c, y_proba)
roc_auc = auc(fpr, tpr)

plt.subplot(1, 2, 2)
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.tight_layout()
plt.show()



# %%

# REGRESSION COMPARISON
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.model_selection import train_test_split

X_reg = df.drop("Performance Index", axis=1)
y_reg = df["Performance Index"]

# Linear Regression
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_reg, y_reg, test_size=0.25, random_state=42)
lr = LinearRegression()
lr.fit(X_train_r, y_train_r)
y_pred_lr = lr.predict(X_test_r)
r2_lr = r2_score(y_test_r, y_pred_lr)

# Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X_reg)
X_train_p, X_test_p, y_train_p, y_test_p = train_test_split(X_poly, y_reg, test_size=0.25, random_state=42)
```

```
poly_model = LinearRegression()
poly_model.fit(X_train_p, y_train_p)
y_pred_poly = poly_model.predict(X_test_p)
r2_poly = r2_score(y_test_p, y_pred_poly)

#  CLASSIFICATION COMPARISON

# Binary classification
y_cls = (df["Performance Index"] > df["Performance Index"].median()).astype(int)
X_cls = df.drop("Performance Index", axis=1)

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_cls, y_cls, test_size=0.25, random_state=42)

# Logistic Regression
log_model = LogisticRegression()
log_model.fit(X_train_c, y_train_c)
roc_log = roc_auc_score(y_test_c, log_model.predict_proba(X_test_c)[:, 1])

# Random Forest
rf_model = RandomForestClassifier()
rf_model.fit(X_train_c, y_train_c)
roc_rf = roc_auc_score(y_test_c, rf_model.predict_proba(X_test_c)[:, 1])

# SVM
svm_model = SVC(probability=True)
svm_model.fit(X_train_c, y_train_c)
roc_svm = roc_auc_score(y_test_c, svm_model.predict_proba(X_test_c)[:, 1])

# SELECT BEST MODELS
```

```
# Regression selection
best_reg_model = "Polynomial Regression" if r2_poly > r2_lr else "Linear Regression"

# Classification selection
auc_scores = {
    "Logistic Regression": roc_log,
    "Random Forest": roc_rf,
    "SVM": roc_svm
}
best_cls_model = max(auc_scores, key=auc_scores.get)

# CLUSTERING COMPARISON

from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score

X_scaled = StandardScaler().fit_transform(X_cls)

kmeans = KMeans(n_clusters=2, random_state=42).fit(X_scaled)
dbscan = DBSCAN(eps=1.5, min_samples=5).fit(X_scaled)
agg = AgglomerativeClustering(n_clusters=2).fit(X_scaled)

ari_scores = {
    "KMeans": adjusted_rand_score(y_cls, kmeans.labels_),
    "DBSCAN": adjusted_rand_score(y_cls, dbscan.labels_),
    "Agglomerative": adjusted_rand_score(y_cls, agg.labels_)
}
```

```
best_cluster_model = max(ari_scores, key=ari_scores.get)

# %%
# Feature Importance Plot
importances = best_rf.feature_importances_
features = df.drop('Performance Index', axis=1).columns
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10, 5))
sns.barplot(x=importances[indices], y=features[indices])
plt.title("Feature Importance - Random Forest")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()



# %%
# Use this block and DO NOT overwrite X_train_c, y_train_c again
y_cls = (df["Performance Index"] >= 60).astype(int)
X_cls = df.drop("Performance Index", axis=1)
X_scaled = StandardScaler().fit_transform(X_cls)

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_scaled, y_cls, test_size=0.2, random_state=42)

# Fit model
grid_search.fit(X_train_c, y_train_c)
best_rf = grid_search.best_estimator_
y_pred_best_rf = best_rf.predict(X_test_c)
```

```
# Evaluation block (now this will work correctly)
train_acc = accuracy_score(y_train_c, best_rf.predict(X_train_c))
test_acc = accuracy_score(y_test_c, y_pred_best_rf)

print(f"Train Accuracy: {train_acc:.4f}")
print(f"Test Accuracy : {test_acc:.4f}")

if abs(train_acc - test_acc) < 0.05:
    print(" Model is a good fit.")
elif train_acc > test_acc:
    print(" Possible overfitting detected.")
else:
    print(" Possible underfitting detected.")



# %%
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Fit a basic decision tree
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train_c, y_train_c)

# Visualize it
plt.figure(figsize=(20, 10))
plot_tree(dt, feature_names=df.drop("Performance Index", axis=1).columns,
          class_names=["Fail", "Pass"], filled=True, rounded=True)
plt.title("Decision Tree Visualization")
plt.show()
```

```
# %%
# Extract one tree from the forest
from sklearn.tree import plot_tree

single_tree = best_rf.estimators_[0]

plt.figure(figsize=(20, 10))
plot_tree(single_tree,
          feature_names=df.drop("Performance Index", axis=1).columns,
          class_names=["Fail", "Pass"], filled=True, rounded=True)
plt.title("Random Tree from Random Forest")
plt.show()


# %%
importances = best_rf.feature_importances_
features = df.drop('Performance Index', axis=1).columns
indices = np.argsort(importances)

plt.figure(figsize=(10, 6))
sns.barplot(x=importances[indices], y=features[indices])
plt.title("Random Forest - Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()


# %%
```

```
for i, tree in enumerate(best_rf.estimators_[:3]):
    plt.figure(figsize=(16, 8))
    plot_tree(tree,
              feature_names=df.drop("Performance Index", axis=1).columns,
              class_names=["Fail", "Pass"], filled=True, rounded=True)
    plt.title(f"Tree {i+1} from Random Forest")
    plt.show()



# %%
print(" Final Model Selection Summary")
print("-" * 50)
print(" Best Performing Models Based on Evaluation Metrics:")
print(f"   • Regression      : {best_reg_model} (R² = {max(r2_lr, r2_poly):.4f})")
print(f"   • Classification  : {best_cls_model} (AUC = {auc_scores[best_cls_model]:.4f})")
print(f"   • Clustering      : {best_cluster_model} (ARI = {ari_scores[best_cluster_model]:.4f})")
print("-" * 50)

# Optional justification text
if best_reg_model == "Linear Regression":
    print(" Linear Regression outperformed Polynomial Regression in R² score.")
else:
    print(" Polynomial Regression achieved higher R² than Linear Regression.")

if best_cls_model == "Logistic Regression":
    print(" Logistic Regression had the highest AUC among classifiers.")
elif best_cls_model == "Random Forest":
    print(" Random Forest performed best in classification (highest AUC).")
else:
```

```
    print(" SVM achieved the best classification AUC.")

if best_cluster_model == "KMeans":
    print(" K-Means had the highest ARI among clustering methods.")
elif best_cluster_model == "Agglomerative":
    print(" Agglomerative (Hierarchical) Clustering had the best ARI score.")
else:
    print(" DBSCAN showed the best clustering ARI score.")



# %%
sample_pass = {
    "Hours Studied": 7,
    "Previous Scores": 70,
    "Extracurricular Activities": 1,  # 1 = Yes
    "Sleep Hours": 5,
    "Sample Question Papers Practiced": 3
}

sample_fail = {
    "Hours Studied": 6,
    "Previous Scores": 70,
    "Extracurricular Activities": 1,  # 0 = No
    "Sleep Hours": 7,
    "Sample Question Papers Practiced": 2
}

def test_prediction(sample_input):
    input_df = pd.DataFrame([sample_input])
    scaled_input = scaler.transform(input_df)
```

```
    prediction = best_rf.predict(scaled_input)[0]
    probability = best_rf.predict_proba(scaled_input)[0][1]
    print(f"Prediction: {'Pass' if prediction == 1 else 'Fail'}")
    print(f"Probability of Passing: {probability:.2f}\n")

print(" Testing Sample PASS Input:")
test_prediction(sample_pass)

print(" Testing Sample FAIL Input:")
test_prediction(sample_fail)
```

Code for Front End

```
import streamlit as st
import pandas as pd
import numpy as np
import warnings
import os
import joblib

# Load trained model and scaler
model = joblib.load("best_rf_model.pkl")
scaler = joblib.load("scaler.pkl")

# CSV path for saving input
csv_path = "user_inputs.csv"
```

```
st.title("Student Performance Predictor")

# Form Input
with st.form("prediction_form"):
    hours = st.slider("Hours Studied", 0, 10, 5)
    prev_score = st.number_input("Previous Scores", 0, 100, 70)
    activities = st.selectbox("Extracurricular Activities", ["No", "Yes"])
    sleep = st.slider("Sleep Hours", 0, 12, 6)
    papers = st.slider("Sample Papers Practiced", 0, 10, 3)

    submit = st.form_submit_button("Predict")

if submit:
    activity_val = 1 if activities == "Yes" else 0

    input_data = pd.DataFrame([{
        "Hours Studied": hours,
        "Previous Scores": prev_score,
        "Extracurricular Activities": activity_val,
        "Sleep Hours": sleep,
        "Sample Question Papers Practiced": papers
    }])

    scaled_input = scaler.transform(input_data)
    prediction = model.predict(scaled_input)[0]
    prob = model.predict_proba(scaled_input)[0][1]

    result = " 🎉 Pass" if prediction == 1 else " Fail"
    st.success(f"Prediction: {result} ")
```

```
input_data["Prediction"] = result
if os.path.exists(csv_path):
    existing = pd.read_csv(csv_path)
    updated = pd.concat([existing, input_data], ignore_index=True)
else:
    updated = input_data
updated.to_csv(csv_path, index=False)
st.info("Input saved to CSV!")
```